

CCS 3105: Systems Programming

Week 5 Lab Report

Process Creation, Process Communication and Process Management

Name: David Mujabi

Reg. No.: C026-01-0802/2024

Date: 13/08/2026

0.1. 1. Objectives

The aim of this session was to move from single programs to real multiprocessing on Linux. By the end of the practical I was expected to create parent and child processes with `fork()`, synchronise them with `wait()`, replace a process image with the `exec()` family, communicate using anonymous pipes and named pipes (FIFOs), redirect standard input and output with `dup2()`, and monitor and terminate processes with `ps` and `kill`.

0.2. 2. Procedure

I began with `fork()`. I compiled `fork_review.c` and ran it several times, noting that the parent and child printed in a changing order. Next I ran the parent-child example where the child calls `sleep(4)`; the parent printed "Parent resumes." only after the child finished, because it called `wait(NULL)`. For `exec()` I wrote a program printing "Before exec" and then calling `execl("/bin/date", "date", NULL)`; "After exec" never printed, and repeating the exercise with `pwd` and `ls -l` showed the same behaviour.

For IPC I created a pipe with `pipe(fd)`, forked, had the parent `write()` "Hello Child" and the child `read()` and print it. I then modified the program so the parent sent my name, registration number and course. For redirection I opened `output.txt`, called `dup2(fd, STDOUT_FILENO)` and verified the text was in the file; I repeated this for `stdin` by reading three lines of `input.txt` with `scanf()`. I ran a program containing `sleep(20)`, located it with `ps -ef`, and killed it with `kill PID`. Finally I created `mypipe` with `mkfifo` and ran `cat > mypipe` and `cat < mypipe` in two terminals.

0.3. 3. Results

The table below records the outputs I observed.

Task	Command/Code	Result
Process creation	<code>fork()</code>	Parent and child both print; order changes between runs
Synchronisation	<code>wait(NULL)</code>	Parent blocks until the child's 4-second sleep ends
Replace image	<code>execl("/bin/date", ...)</code>	Date printed; "After exec" never appears
Anonymous pipe	<code>pipe() + write()/read()</code>	Child prints "Child received: Hello Child"

Task	Command/Code	Result
Redirect stdout	<code>dup2(fd, STDOUT_FILENO)</code>	<code>cat output.txt</code> shows “Redirected Output”
Redirect stdin	<code>dup2(fd, STDIN_FILENO)</code>	<code>scanf()</code> reads the 3 lines of <code>input.txt</code>
Process monitor	<code>ps -ef + kill PID</code>	Process holding <code>sleep(20)</code> found and terminated
Named pipe	<code>mkfifo mypipe</code>	Two terminals exchange messages in real time

0.4. 4. Discussion

The `fork()` exercise made the process model concrete. `fork()` returns once in each process but with different values: zero in the child, the child’s PID in the parent, and -1 on failure. Because the two processes run independently, the scheduler decides the order — running the program repeatedly confirmed that nothing is guaranteed. This is why `wait()` matters: when a child exits, its status stays in the kernel until collected, and a parent that never calls `wait()` leaves a zombie that still holds a PID. `exec()` explains why “After `exec`” never appeared — instead of starting a new process, `execl()` replaces the **current** image with the named program, same PID but new code, so control never returns. `fork()` and `exec()` are therefore complementary.

The pipe exercises showed that IPC is one-way. `pipe()` returns two descriptors, `fd[1]` for writing and `fd[0]` for reading, into a shared buffer that `fork()` duplicates into both processes. Each process must close the end it does not use — the child closes `fd[1]` so it sees EOF when the parent finishes, and the parent closes `fd[0]` so the pipe is not held open. This is what `dup2()` exploits for redirection: copying a descriptor onto `STDOUT_FILENO` makes `printf()` write into `output.txt` instead of the screen. A FIFO is a named pipe that appears as a file (`mkfifo`), so unrelated processes can use it, unlike an anonymous pipe, which only exists while a process holds it.

0.5. 5. Challenges Encountered

My first `mkfifo` test blocked because I started the writer before any reader existed — writes to a FIFO wait until the other end opens. I also got a broken-pipe style failure when one process closed its end early, which taught me to close only the unused descriptor, not the whole pipe. During monitoring I had to scan `ps -ef` carefully to pick the right PID, and I confirmed the difference between `kill` (SIGTERM, allows cleanup) and `kill -9` (SIGKILL, uncatchable).

0.6. 6. Lessons Learned

The main idea I will keep is that processes are composable building blocks: `fork()` creates them, `exec()` reimages them, `wait()` synchronises them, and pipes move data between them. Always closing the unused pipe end and always reaping children with `wait()` is what separates a working program from one that hangs or leaks zombies.